

✓ Mizan' est 100% opérationnel après la mise à jour 3.7.4.4, n'hésitez pas à reporter les bugs dans le salon associé ! [Rejoindre le Discord](#) ✕

## API Lua

Référence complète des fonctions Lua exposées à tes scripts MizanBot. Tout ce qu'un script peut faire — récolte, combat, déplacement, HDV, trajet déclaratif — avec exemples.

Un script définit des globals de configuration (MAX\_PODS, ELEMENTS\_TO\_GATHER, MIN\_MONSTERS, MAX\_MONSTERS...) et une fonction `move()`, plus, en option, `bank()` et des callbacks (`onTurn`, `onPlacement`, `onFightEnd`, `stopped`).

Le moteur appelle `move()` en boucle, à chaque tick ; si les pods atteignent MAX\_PODS et que `bank()` existe, il joue `bank()` à la place. `move()` devient facultative si le script ne définit que des hooks (combat, paquets, MP).

`move()` peut soit renvoyer une table = trajet déclaratif (liste d'étapes { `map`, `path`, `gather`, `fight`, ... }) que le moteur déroule ; soit exécuter des verbes impératifs (`goToCellId`, `gatherAll`, `fight`...) et renvoyer `false` pour arrêter le script.

L'API expose des verbes globaux que tu appelles directement (`goToCellId`, `fight`, `gatherAll`...) et des MODULES en syntaxe colon (carte suivante). L'appel point (`map.getResources()`) marche aussi. Les noms sont tolérants comme chez SnowBot : 1re lettre majuscule ou minuscule (`sale:sellItem` = `sale:SellItem`), `snake_case` (`map:change_map` = `map:changeMap`), casse ignorée en dernier recours.

Un module regroupe les getters et actions d'un domaine — ex. `combat:enemies()`, `map.getResources()`, `team:members()`. Modules disponibles : `map:`, `combat:`, `fightAction:`, `fightChallenge:`, `fightCharacter:`, `fightBasic:`, `fightDebug:`, `fightSlave:`, `team:`, `inventory:`, `bank:`,

exchange:, trade:, craft:, merchant:, zaap:, zaapi:, job:, sale: (HDV), npc:, chat:, character:, mount:, stats:, quest:, treasure:, data: (alias d2data:), memory:, sharedMemory:, http:, developer:, global:, config:, snowbotController:.

Lua MoonSharp, un hôte par personnage : string, table, math, bit32, coroutine, pcall/error, métatables, os.time / os.clock / os.date et json sont disponibles ; io et os.execute ne le sont pas. load / loadstring compile un TEXTE dans le même bac à sable et dofile(nom) exécute un fichier de scripts/lib puis scripts, trouvé par son NOM (jamais un chemin disque) — voir Plateforme.

Les listes sont 1-based ; -1 = valeur inconnue (PV, XP, kamas...), 0 = id absent. Tout est journalisé dans la console du bot et dans %APPDATA%/MizanBot/lua/<perso>.log.

### Actions bloquantes

Les déplacements et les actions marquées « bloquant » attendent la résolution serveur (arrivée / refus / délai) avant de rendre la main ; les envois simples (MP, dépôt sur un coffre déjà ouvert, destruction d'objet...) rendent la main dès l'envoi. L'arrêt (stopScript(), bouton Stop, planning) interrompt proprement le verbe en cours.

### Où appeler les verbes d'action

Les verbes existent dès la première ligne, mais appelle les verbes d'ACTION dans move(), bank() ou un callback : au top-level, l'état du perso (nom, carte) peut ne pas encore être connu.

### Droits serveur (permissions)

Les actions sont gâtées par les permissions de ta licence (liste dans « Mon compte »), vérifiées au point d'action. Sans la permission, le verbe renvoie false (ou 0) et l'explique dans la console — ce n'est pas un bug de script. Les lectures pures ne sont pas contrôlées.

Récolte auto : récolte, interactifs, inventaire, banque, coffres, échanges, craft, chasse au trésor, équipement, validation de quête, emote, statut. Combat auto : lancer / jouer un combat, verrous, abandon, expulsion, variantes, réglages auto, caractéristiques.

Multi : groupe Dofus, team:\*, joinBoss, mode mule, setPrivateTeam, executeLUA, lancement

d'un 2e compte. Hôtel de Vente : goToHdv, goToHdvBuy, actions sale:\*, npc:npc(id, 5 | 6).  
Réponse automatique : MP, canaux, chat:\*, sendWebhook, http:\*, getRequest / postRequest, MODERATOR\_WEBHOOK.

Un script SnowBot tourne SANS modification : ses tables existent (global:, config:, snowbotController:, npc:, chat:, craft:, merchant:, mount:, treasure:, d2data:, fightCharacter:, fightBasic:, fightDebug:, fightSlave:...), les noms de messages Dofus 2 sont acceptés et les callbacks phenix, prefightManagement et challengeManagement sont appelés.

Rechercher une fonction...

Configuration & points d'entrée

Cycle de vie & callbacks

Journal / console

Messagerie, notifications & statut

Contrôle du script

État du personnage

Vie, énergie & régénération

Déplacement & voyage

Interactifs & récolte

Combat (verbes globaux)

Inventaire (verbes globaux)

Havre-sac

PNJ / dialogue

Banque (coffre ouvert)

HdV — table sale:

Module map: — carte courante

Module map: — interactifs & géométrie

Module map: — respawn, coffres & maisons à code

Module combat: — lire le plateau & jouer le tour

Module combat: — réglages, défis, verrous & bilan

Tables fightAction: & fightChallenge: (compat SnowBot)

Tables fightCharacter: fightBasic: fightDebug: & fightSlave: (compat SnowBot)

Tables rightCharacter, rightData, rightDebug, & rightError (compat SnowBot)

Module team: — groupe Dofus & équipe

Module inventory:

Module bank:

Module exchange: (compat SnowBot)

Module trade: — échange entre joueurs

Module merchant: — marchands (compat SnowBot, neutre)

Modules zaap: & zaapi: — téléport

Métiers & craft (job:, craftItem)

Module craft: — craft pas à pas (compat SnowBot)

Module npc: — PNJ, banque & HDV (compat SnowBot)

Module chat: — messages (compat SnowBot)

Module character:

Module mount: — montures (valeurs neutres)

Module stats: — compteurs

Module quest: — quêtes

Module treasure: — chasse au trésor (compat SnowBot)

Module data: (alias d2data:) — données statiques du jeu

Module d2data: (compat SnowBot)

Modules memory: & sharedMemory: — état persistant

Table global: — verbes SnowBot

Module config: — réglages de script (compat SnowBot)

Module snowbotController: — comptes de l'app (compat SnowBot)

Plateforme : hooks de paquets, modules, fichiers & bascule de script

Module developer: — paquets (compat SnowBot)

Modules http: & developer: — requêtes web

Trajet déclaratif (move() / bank())

Compatibilité SnowBot

Non supporté

## Configuration & points d'entrée

Globals lus par le moteur, et les fonctions qu'il attend. Absents = comportement par défaut (un

ancien script se comporte comme avant). Booléens : true/false, nombre (≠ 0 = vrai) ou chaîne "1"/"true".

FIGHT\_LOCK\_\*, IGNORE\_REQUEST\_\* (dont GUILD), MODERATOR\_WEBHOOK, MODERATOR\_STOP, PLANNING et FIGHT\_KICK sont relus pendant le run : tu peux les changer dans move(). Ils ne valent que pendant que le script tourne (pas pour une commande /cmd). MAX\_PODS, TURN\_TIME\_LIMIT et SNOWBOT\_TIMEOUT sont lus au démarrage ; les autres (dont GATHER et MONSTERS\_AMOUNT) à chaque usage.

Quand les pods atteignent MAX\_PODS : OPEN\_BAGS puis AUTO\_DELETE, réévaluation des pods, puis bank() seulement si c'est encore plein. Rejouée au plus toutes les 20 s tant que les pods restent pleins. Sans bank(), un avertissement « pods pleins » est journalisé et move() continue.

#### Scripts SnowBot : MAX\_PODS et MAX\_MONSTERS

SnowBot n'a pas de MAX\_PODS : son seuil de banque est un réglage d'interface (90 % par défaut, et un chef banque dès qu'UN membre l'atteint). Un script SnowBot porté tel quel part donc en banque à 95 % (défaut MizanBot, par bot) : ajoute MAX\_PODS = 90. De même, MAX\_MONSTERS absent vaut 8 chez SnowBot et 999 ici : déclare MAX\_MONSTERS = 8 (ou appelle config:setMaxMonsters, qui écrit ce global). Aucun verbe de config: ne pose MAX\_PODS.

| GLOBAL             | TYPE   | DÉFAUT | RÔLE                                                                                                                 |
|--------------------|--------|--------|----------------------------------------------------------------------------------------------------------------------|
| MAX_PODS           | number | 95     | Seuil (% de pods) au-dessus duquel le moteur bascule sur bank() au lieu de move(). Lu au démarrage.                  |
| ELEMENTS_TO_GATHER | table  | {}     | Liste d'ElementTypeld à récolter. Vic = toutes les VRAIES ressources (jamais les portes, zaaps ou actions passives). |
| GATHER             | table  | absent | Nom SnowBot de ELEMENTS_TO_GATHER, lu seulement quand ELEMENTS_TO_GATHER n'est pas une table.                        |
|                    |        |        | Taille MIN d'un groupe de monstres (somme des monstres) pour                                                         |

|                         |             |        |  |                                                                                                                                                                                                           |
|-------------------------|-------------|--------|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MIN_MONSTERS            | number      | 1      |  | l'attaquer.                                                                                                                                                                                               |
| MAX_MONSTERS            | number      | 999    |  | Taille MAX d'un groupe de monstres pour l'attaquer.                                                                                                                                                       |
| FORBIDDEN_MONSTERS      | table       | {}     |  | Noms ou genericId : un groupe qui contient l'un d'eux est ignoré.                                                                                                                                         |
| FORCE_MONSTERS          | table       | {}     |  | Non vide = n'attaquer QUE les groupes contenant l'un de ces monstres (noms ou genericId).                                                                                                                 |
| MONSTERS_AMOUNT         | table       | {}     |  | { {gid, min, max}, ... } (SnowBot config:setAmountOfSpecificMonste : un groupe qui contient ce monstre est écarté si son nombre est hors [min, max] ; un groupe qui n'en contient pas n'est pas concerné. |
| TURN_TIME_LIMIT         | number (ms) | 20000  |  | Budget de onTurn / onPlacement (borné 3000-25000). Dépassé : le tour est fini d'office (ou « prêt » envoyé). Lu au démarrage.                                                                             |
| OPEN_BAGS               | bool        | false  |  | Pods pleins → ouvrir tous les sacs d ressources avant bank() (et même sans bank()).                                                                                                                       |
| AUTO_DELETE             | table       | absent |  | GID détruits quand les pods sont pleins, avant bank() (stacks du sac uniquement, jamais un objet équipé)                                                                                                  |
| FIGHT_LOCK_SPEC         | bool        | false  |  | À chaque combat : interdit les spectateurs (chef seulement).                                                                                                                                              |
| FIGHT_LOCK_JOIN         | bool        | false  |  | À chaque combat : ferme le combat aux arrivants (en équipe, attend d'abord les followers, 8 s max).                                                                                                       |
| FIGHT_LOCK_PARTY        | bool        | false  |  | À chaque combat : combat réservé : groupe.                                                                                                                                                                |
| IGNORE_REQUEST_EXCHANGE | bool        | false  |  | Refus auto (~1 s) des demandes d'échange des joueurs hors app ; jamais pendant un dialogue PNJ / banque / HDV / échange.                                                                                  |

|                      |                       |                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------|-----------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IGNORE_REQUEST_DUEL  | bool                  | false                          | Refus auto des défis (duels) des joueurs hors app.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| IGNORE_REQUEST_GUILD | bool                  | false                          | Refus auto (~1–2 s) des invitations de guilde, sauf celles d'un perso d'un c TES comptes enregistrés (SnowBot ne refusait que les inconnus). Opt-ir SnowBot les refuse par défaut.                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| MODERATOR_WEBHOOK    | string                | absent                         | URL https:// de webhook Discord : tout popup de modérateur y est rela immédiatement (permission « Réponse automatique »).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| MODERATOR_STOP       | bool                  | false                          | Popup de modérateur reçu → arrêt c script à la prochaine itération (jamai au milieu d'un trajet ; aussi pendant l'attente de map:forceFight) + planning en pause pour ce perso.                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| PLANNING             | table                 | absent                         | Heures (0-23, heure locale du PC) c DÉCONNEXION, sémantique SnowB (indépendant du planning de l'ongle contrôlées hors combat à chaque itération (et pendant l'attente de map:forceFight) : à une heure listée, bot se déconnecte (chef : toute l'équipe ; un suiveur l'ignore), se reconnecte à la 1re heure libre (+ 5 à 90 s) et reprend le script. Demande refusée (file d'actions du compte pleine) = nouvel essai à l'itération suivante. D'ici la reconnexion, la connexion auto du planning de l'onglet ne relance pas le compte ; u Arrêter / Lancer manuel l'annule (script non repris). 24 heures listées ignoré. |
| SNOWBOT_TIMEOUT      | number (tours de 6 s) | absent (réglage du bot, 180 s) | Seuil anti-blocage de CE run, en TOURS SnowBot de 6 s (pas en secondes) : seuil = valeur × 6 s, 60 minimum (50 ≈ 5 min ; 120, le défaut SnowBot, ≈ 12 min). Remplace le réglage anti-blocage du bot le temp du run. Sans effet si l'anti-blocage e                                                                                                                                                                                                                                                                                                                                                                          |

|                     |                         |                 |                                                                                                                                                                                                                                                                                                                           |
|---------------------|-------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                     |                         |                 | désactivé dans les réglages du bot.<br>Lu au démarrage.                                                                                                                                                                                                                                                                   |
| FIGHT_KICK          | number<br>(ms)          | absent<br>(off) | Placement : le chef (ou un solo) expulse chaque joueur étranger à l'équipe présent depuis ce délai (une tentative par joueur et par placement permission « Combat auto »), y compris pendant un combat lancé par fight(), fightCell(), une étape fight / forcefight ou map:forceFight. Opt-in SnowBot expulse par défaut. |
| MIN_LIFE_PERCENT    | number                  | 0 (off)         | Avant d'engager un combat (fight, fightCell, étape fight/forcefight) : régénère si PV% < seuil. PV inconnu = pas de blocage.                                                                                                                                                                                              |
| REGEN_UP_TO_PERCENT | number                  | 100             | Cible de la régénération automatique (jamais sous le seuil déclencheur).                                                                                                                                                                                                                                                  |
| REGEN_FOOD          | table<br>(ou<br>number) | { }             | GlD de consommables utilisés d'abord, dans l'ordre de la liste (chacun confirmé par la hausse des PV).                                                                                                                                                                                                                    |
| REGEN_SIT           | bool                    | true            | S'asseoir pendant l'attente de régénération (plus rapide).                                                                                                                                                                                                                                                                |
| REGEN_SIT_EMOTE     | number                  | 1               | Id de l'emote envoyée pour s'asseoir.                                                                                                                                                                                                                                                                                     |
| REGEN_MAX_WAIT      | number<br>(s)           | 600             | Plafond d'attente de la régénération automatique (min 15).                                                                                                                                                                                                                                                                |

```

MAX_PODS = 90
ELEMENTS_TO_GATHER = { 38 } -- Blé
FORBIDDEN_MONSTERS = { "Bouftou Royal" } -- nom ou genericId
OPEN_BAGS = true
AUTO_DELETE = { 311, 1984 }
MIN_LIFE_PERCENT = 50

function move()
    -- soit renvoyer une table d'étapes (trajet déclaratif),
    -- soit piloter impérativement + return false pour stopper
end

```

Copier

```
function bank()          -- optionnelle : appelée quand les pods atteignent MAX_P
    return { --[[ étapes vers la banque ]] }
end
```

## Cycle de vie & callbacks

Fonctions GLOBALES que ton script définit (au top-level) et que le moteur appelle. Toutes optionnelles sauf `move()` — un script sans `move()` est accepté s'il définit un hook de combat, de paquet ou de MP (il ne pilote alors que ça). Noms SnowBot reconnus : `fightManagement`, `phenix`, `prefightManagement`, `challengeManagement`.

1) relecture des globals `FIGHT_LOCK_*`, `IGNORE_REQUEST_*`, `MODERATOR_*` ; 2) mort → résurrection au Phénix (option `AutoRevive` du profil, ou `phenix()` du script), prioritaire ; 3) combat → le trajet est en pause (`FIGHT_KICK` appliqué au placement, aussi pendant un `fight()` ou un combat de trajet), `onPlacement` (ou `prefightManagement`) / `onTurn` sont appelés quand le moteur leur tend la main, `challengeManagement` dès que les défis sont proposés ; 4) fin de combat → `onFightEnd(result)`, une fois ; puis, à chaque itération hors combat, contrôle de `PLANNING` ; 5) pods ≥ `MAX_PODS` → `OPEN_BAGS` / `AUTO_DELETE` puis `bank()` ; 6) sinon `move()`. `move()` qui renvoie une table = trajet déclaratif ; `false` = fin du script ; autre chose = rappelée (~200 ms).

### Règles de `onTurn`

Si `onTurn` existe, il REMPLACE l'auto-combat du profil (sauf `combat:setAutoFight(true)` ou `combat:playTurn()`). Budget `TURN_TIME_LIMIT` : dépassé, le tour est fini d'office et les actions suivantes sont refusées ; un `delay()` dans le hook s'interrompt dès que le tour est perdu. Erreur Lua sans aucune action + auto-combat du profil actif : le moteur joue ce tour. Tacle : quitter une case au contact d'un ennemi est refusé par le serveur, à toi de le gérer. Garde le « prêt » automatique du client Dofus désactivé.

`mule_lost(carteDuChef)` n'est JAMAIS appelé (le suivi d'équipe MizanBot gère un suiveur perdu) et il n'existe pas de script « contrôleur » sans compte (`run()`). `stopped(reason)` est appelé à CHAQUE fin de `run`, alors que SnowBot ne l'appelait quasiment jamais.

`onTurn()`

`fightManagement()`

Appelé UNE fois par tour de ton perso en combat, sur le thread du script (même pendant un `fight()` ou un `gatherAll()`). Fin de tour automatique au retour, sauf si le script l'a déjà finie. `fightManagement()` est l'alias `SnowBot`, utilisé si `onTurn` est absent.

```
function onTurn()  
    local cible = combat:enemies()[1]  
    if cible and combat:canCast(161, cible.cell) then  
        combat:castOnCell(161, cible.cell)  
    end  
end    -- fin de tour automatique
```

Copier

### `onPlacement()`

Appelé une fois par combat à l'arrivée des cases de placement (même budget que `onTurn`). « Prêt » automatique au retour (le chef attend brièvement ses followers), sauf si le script a appelé `combat:ready(...)` ou si la fonction renvoie `false`. Doit rendre la main vite.

```
function onPlacement()  
    local cases = combat:placementCells()  
    if #cases > 0 then combat:place(cases[1].cell) end  
    combat:lock(true, false, false)    -- pas de spectateurs  
end
```

Copier

### `onFightEnd(result)`

Appelé une fois après chaque combat, hors combat, carte posée (entre deux `move()`). `result` = même bilan que `combat:lastFightResult()` : `won`, `lost`, `outcome`, `loot`, `challenges`, `durationMs`...

```
function onFightEnd(r)  
    if r.won then printSuccess("Victoire, " .. #r.loot .. " lignes de butin")  
    else printWarning("Combat perdu") end  
end
```

Copier

### `stopped(reason)`

Appelé une fois à la fin du run, APRÈS que le moteur a repris la main (budget 2 s). Les verbes bloquants y lèvent une erreur : n'utilise que `print*`, `sendWebhook`, `memory:*`, `leaveDialog` et des lectures. `reason` ∈ `ended`, `error`, `user`, `script`, `stuck`, `planning`, `remote`, `team`, `levelcap`, `license`, `moderator`, `restart`, `shutdown`, `disconnected` (session de jeu perdue, bot gardé dans la

liste : MITM — Dofus.exe reste ouvert, déconnexion par le serveur, session quittée sans retour en jeu sous 1 min, ou relance du client par Reconnecter — ou mode Socket déconnecté par le serveur, session en pause sans reconnexion automatique ; le script est conservé pour le bouton Reconnecter), socket (mode Socket : connexion perdue ou coupée par Reconnecter, script suspendu puis repris automatiquement après la reconnexion). Après un « Déconnecté », le bouton Reconnecter (bandeau de l'onglet Bots, carte du compte dans l'onglet Comptes) reconnecte le compte tout de suite, sur le même personnage, sans le retirer de la liste des bots (MITM : Dofus relancé ; Socket : session reconnectée), et reprend le script qui tournait (relance depuis le début : garder l'état dans memory:) ; jamais automatique après une déconnexion par le serveur.

```
function stopped(reason)
    printWarning("Arrêt : " .. reason)
    memory:set("dernierArret", reason)
end
```

Copier

**onPrivateMessage**(from, text, senderId)

Appelé à chaque MP reçu : pseudo de l'expéditeur, texte, id.

```
function onPrivateMessage(from, text)
    notify("MP de " .. from .. " : " .. text)
end
```

Copier

**messagesRegistering**()

Appelée une fois après le top-level : c'est l'endroit où enregistrer les hooks de paquets avec registerMessage (voir la catégorie Plateforme).

```
function messagesRegistering()
    registerMessage("ChatChannelMessageEvent", function(p)
        print((p.sender_name or "?") .. " : " .. (p.content or ""))
    end)
end
```

Copier

**phenix**() → table

Compat SnowBot. Si le script la définit, elle pilote le FANTÔME, même si l'option AutoRevive

du profil est coupée (la tombe, elle, est libérée par le moteur) : appelée à chaque itération tant que le perso est fantôme, elle renvoie un trajet dont l'étape de la carte courante est jouée. Clé phenix = <cellule de la statue> ("c1|c2" = une au hasard, comme door) : l'utiliser ; sinon path / door / custom comme un trajet normal. Étape SANS effet (même carte, toujours fantôme : pas de statue à cette cellule, étape terminale, porte absente, custom qui ne bouge pas...) : pause d'environ 1,5 s avant le rappel ; après 3 passages sans effet sur la même carte, le moteur reprend la résurrection (avertissement journalisé) jusqu'au prochain changement de carte. Aucune étape pour cette carte (ou erreur Lua, ou pas une table) : le moteur reprend et voyage vers le Phénix indiqué par le serveur.

```
function phenix()  
  return {  
    { map = CARTE_AVANT_PHENIX, path = "top" },  
    { map = CARTE_PHENIX, phenix = CELLULE_STATUE }, -- utilise la sta  
  }  
end
```

Copier

### **prefightManagement**(mesCases, casesAdverses)

Compat SnowBot, si onPlacement est ABSENT : une NOTIFICATION au début du placement (sur le thread du script, budget TURN\_TIME\_LIMIT : dépassé, « prêt » d'office), pas une prise en main. Arguments au format SnowBot { [cellule] = occupant | -1 } : mesCases = cases de placement de ton équipe + cases de tes combattants vivants ; casesAdverses = idem pour l'équipe adverse ; -1 = case libre, occupant = id du combattant (valeur ABSOLUE pour un monstre ou une invocation : -1 est réservé aux cases libres). Avant l'appel, le moteur attend (≤ ~1,6 s) qu'au moins un ennemi soit connu. Au retour, si l'option « pré-combat auto » du profil Combat est active, le moteur fait son pré-combat : placement selon le style — sauf si le script s'est placé lui-même pendant l'appel (combat:place / fightAction:chooseCell réussi) —, attente bornée des followers, défi selon le mode du profil, puis « prêt ». Option coupée : ni placement ni « prêt » du moteur, comme sans callback. return false = contrat d'onPlacement : ni placement ni « prêt » du moteur (à toi d'appeler combat:ready()). Erreur Lua : journalisée, le moteur place quand même.

```
function prefightManagement(mesCases, casesAdverses)  
  for cell, occupant in pairs(mesCases) do  
    if occupant == -1 then combat:place(cell) break end -- case libre  
  end  
end
```

Copier

## challengeManagement(nbDefis, estSuiveur)

Compat SnowBot (si onPlacement est absent). Sa présence passe le choix du défi en mode « script » (permission « Combat auto »). Appelée UNE fois par placement, sur le thread du script (pompe des hooks) : chef ou solo, dès que les défis proposés sont arrivés (nbDefis = leur nombre ; aucun défi = pas d'appel) — au retour, le pré-combat du moteur (s'il tourne) envoie « prêt » sans attendre la fenêtre de choix de 8 s ; membre d'équipe MizanBot non chef (estSuiveur = true), tout de suite avec nbDefis ≥ 1, pour information (le chef choisit). Choisis avec combat:selectChallenge(id). Les défis sont demandés par le pré-combat du moteur, ou par l'hôte quand celui-ci ne tourne pas (« pré-combat auto » coupé, ou prefightManagement a renvoyé false).

```
function challengeManagement(n, suiveur)
    if suiveur then return end
    local d = combat:challengeProposals()[1]
    if d then combat:selectChallenge(d.id) end
end
```

Copier

```
-- IA de tour scriptée : cible la plus faible, se replace si besoin, lance
TURN_TIME_LIMIT = 15000
local SORT = 161 -- id de TON sort d'attaque (voir combat:getSpell

local function cibleLaPlusFaible()
    local best
    for _, e in ipairs(combat:enemies()) do
        if e.alive and (not best or e.hpPercent < best.hpPercent) then best = e
    end
    return best
end

function onTurn()
    -- PV bas : on laisse le moteur jouer ce tour avec le profil de l'onglet Comb
    if combat:getHp() < combat:getMaxHp() * 0.2 then combat:playTurn() return end

    local cible = cibleLaPlusFaible()
    if not cible then return end

    -- Pas lançable d'ici ? chercher une case atteignable d'où il l'est
    if not combat:canCast(SORT, cible.cell) then
        for _, c in ipairs(combat:reachableCells()) do
            if combat:canCast(SORT, cible.cell, c.cell) then
                combat:moveToCell(c.cell)
            end
        end
    end
end
```

Copier

```

        break
    end
end
end

-- Lancer tant que le sort est disponible (PA, relance, lancers par tour)
for _ = 1, 4 do
    local ok, raison = combat:canCast(SORT, cible.cell)
    if not ok then printMessage("arrêt des lancers, raison " .. raison, "debut")
        combat:castOnCell(SORT, cible.cell)
    end
end

-- PM restants : s'éloigner de la cible (kite simple ; attention au tackle)
local _, pm = combat:pointsLeft()
if pm > 0 then
    local loin, dmax = nil, -1
    for _, c in ipairs(combat:reachableCells()) do
        local d = map:cellDistance(c.cell, cible.cell)
        if d > dmax then loin, dmax = c.cell, d end
    end
    if loin then combat:moveToCell(loin) end
end
end -- retour = fin de tour automatique

function onPlacement()
    local cases = combat:placementCells()
    if #cases > 0 then combat:place(cases[1].cell) end
end -- « prêt » automatique au retour

```

## Journal / console

Tout est journalisé dans la console du bot (catégorie + niveau coloré) et dans le fichier de log du perso.

**print**(message)

Écrit un message d'information dans la console (seul le premier argument est affiché). Les balises <b>...</b> s'affichent en gras.

```
print("Début de la récolte")
```

Copier

**printSuccess**(message)

```
printWarning(message)
```

```
printError(message)
```

Message aux niveaux succès (vert), avertissement (jaune), erreur (rouge).

```
printSuccess("Sac vidé !")  
printWarning("Pods presque pleins")  
printError("Ressource introuvable")
```

Copier

```
printMessage(message [, niveau])
```

Message coloré par niveau nommé. niveau ∈ "info", "green"/"success"/"ok", "yellow"/"warning"/"warn", "red"/"danger"/"error", "grey"/"gray"/"debug". Omis → info.

```
printMessage("J'ai 3 sacs de Millet", "green")
```

Copier

```
printColor(message, "#RRGGBB")
```

Message avec une couleur hexadécimale libre (#RGB accepté). Les deux ordres d'arguments sont acceptés.

```
printColor("Spot rare détecté", "#f9e2af")
```

Copier

## Messagerie, notifications & statut

MP (par pseudo) et messages de canal, webhooks Discord, notifications locales et statut joueur. La table global: (SnowBot) expose aussi lastModeratorMessage, setPrivate, setPrivateTeam, getPlayerStatus et isPrivate ; le module chat: reprend les noms SnowBot (ordre contenu, destinataire).

```
sendPrivateMessage(pseudo, message) → bool
```

Perm.PmRelay

Envoie un MP à un joueur par son pseudo. true si injecté. L'ordre SnowBot (message, pseudo) est corrigé automatiquement quand le message contient un espace et le pseudo aucun.

```
sendPrivateMessage("SuperPote", "j'arrive")
```

Copier

**sendChannelMessage**(canal, message) → bool

Perm.PmRelay

Envoie un message sur un canal : 0 général, 5 commerce, 6 recrutement, 2 guildes... true si injecté.

```
sendChannelMessage(5, "vends Millet x1000")
```

Copier

**sendWebhook**(url, message) → bool

Perm.PmRelay

Envoie message sur un webhook Discord (url https://).

```
sendWebhook(MON_WEBHOOK, getCharacterName() .. " : banque faite")
```

Copier

**notify**(message [, son=true]) → bool

Toast local dans l'app (+ son et centre de notifications). 200 caractères max, au plus 1 toutes les 2 s (false sinon).

```
notify("Archimonstre repéré !")
```

Copier

**setPrivate**([bool=true]) → bool

Perm.Gather

Passe le perso en statut privé (true, par défaut) ou disponible (false).

```
setPrivate()      -- privé
setPrivate(false) -- disponible
```

Copier

**setPrivateTeam**([bool=true]) → number

Perm.Multi

Chef d'équipe seulement : bascule le statut de tous les membres en jeu ; renvoie le nombre

de membres basculés.

**getPlayerStatus()** → **number**

**isPrivate()** → **bool**

Statut joueur courant (1 disponible, 4 privé) / true si privé.

**lastModeratorMessage()** → **table|nil**

Dernier popup de modérateur reçu : { author, content, lockDuration, time, age } ou nil. Voir aussi MODERATOR\_WEBHOOK / MODERATOR\_STOP.

```
local m = lastModeratorMessage()
if m and m.age < 60 then stopScript() end
```

Copier

## Contrôle du script

**delay(ms)**

Pause interruptible de ms millisecondes, avec ±15 % d'humanisation (l'arrêt du script la coupe immédiatement). C'est aussi un point où les hooks de paquets sont servis.

```
delay(1500)
```

Copier

**stopScript()**

Arrête proprement le script (identique au bouton Stop ; stopped(reason) reçoit "script").

```
if getKamas() > 1000000 then stopScript() end
```

Copier

**isScriptPlaying()** → **bool**

true si le script PRINCIPAL du perso tourne (utile depuis une commande /cmd).

## État du personnage

**getMapId()** → **number**

Id de la carte courante (0 si inconnu).

**getCurrentPos()** → **string**

Coordonnées monde "x,y" (chaîne vide si inconnues).

**getCurrentCell()** → **number**

Cellule (0-559) où se trouve le personnage (-1 si inconnue).

**getMapIdAt(x, y)** → **number**

Id de la carte à la coordonnée monde (x, y) sur le MÊME calque que le perso (0 si inconnu).

```
local m = getMapIdAt(-15, 39)
```

Copier

**getPods()** → **number**

**getPodsMax()** → **number**

**getPodsPercent()** → **number**

Pods actuels, max, et pourcentage de remplissage.

```
if getPodsPercent() >= 90 then printWarning("Presque plein") end
```

Copier

**getCharacterLevel()** → **number**

Niveau du personnage (0 si inconnu).

**getCharacterName()** → **string**

Nom du personnage courant (chaîne vide si inconnu).

**getCharacterId()** → **number**

characterId du personnage (aussi character:id()).

**getCharacterXp()** → **number**

**getXpNextLevel()** → **number**

**getXpLevelFloor()** → **number**

XP totale du personnage, XP requise pour le niveau suivant, XP plancher du niveau actuel (-1 si inconnue).

**getServerName()** → **string**

Nom du serveur de jeu.

**getStatus()** → **string**

Activité du bot : "dead", "fight", "gathering", "moving", "ingame", "connecting", "disconnected" (session de jeu perdue, bot gardé dans la liste : MITM — Dofus.exe reste ouvert, déconnexion par le serveur ou session quittée sans retour en jeu sous 1 min — ou mode Socket déconnecté par le serveur, session en pause jusqu'à Reconnecter ; le script en cours est alors arrêté avec stopped("disconnected"), la valeur se lit donc surtout dans stopped() ou "offline". Pendant une reconnexion (Reconnecter, relance du client MITM comprise) : "connecting".

**getJobLevel(jobId)** → **number**

Niveau du métier jobId (0 si jamais reçu / non appris ; récolter ou crafter une fois le met à jour). Voir aussi le module job:.

```
if getJobLevel(2) >= 100 then print("Bûcheron 100") end
```

Copier

**isInFight()** → **bool**

true si le personnage est actuellement en combat.

**msSinceFightEnd()** → **number**

Millisecondes écoulées depuis la fin du dernier combat (-1 si aucun).

**afterFight()** → **bool**

true UNE fois après chaque fin de combat, puis se réarme (parité SnowBot).

```
if afterFight() then print("Retour de combat") end
```

Copier

**getCountGather()** → **number**

**getCountFight()** → **number**

**resetCountGather()**

**resetCountFight()**

Compteurs de récoltes / combats de la session du script (remis à zéro à chaque démarrage) et leur remise à zéro. Voir aussi le module stats:.

## Vie, énergie & régénération

PV réels hors combat (fiche + estimation de la régénération), énergie et mort. La résurrection au Phénix est AUTOMATIQUE côté moteur (option AutoRevive du profil, prioritaire sur move()) : le script n'a rien à coder. Un script SnowBot peut la piloter avec phenix() (voir Cycle de vie & callbacks).

MIN\_LIFE\_PERCENT déclenche la régénération avant chaque combat ; la clé d'étape regeneration la déclenche sur une carte précise du trajet ; regenerate() la lance à la demande. Les réglages REGEN\_\* (voir Configuration) s'appliquent aux trois.

**getLifePoints()** → **number**

**getMaxLifePoints()** → **number**

**getLifePointsPercent()** → **number**

PV actuels / max / pourcentage (-1 si inconnus). Hors combat : estimation tenant compte de la régénération.

```
if getLifePointsPercent() >= 0 and getLifePointsPercent() < 40 then regenerate
```

[Copier](#)

**getEnergyPoints()** → number

**getMaxEnergyPoints()** → number

Points d'énergie actuels / max (-1 si inconnus).

**isDead()** → bool

**getLifeStatus()** → number

isDead() = true si tombe ou fantôme ; getLifeStatus() : 0 vivant, 1 tombe, 2 fantôme.

**getPhoenixMapId()** → number

Carte du Phénix indiquée par le serveur (utile en fantôme).

**regenerate**([minPct [, maxWaitSec]]) → bool

Bloquant

Régénère jusqu'à minPct (défaut MIN\_LIFE\_PERCENT, sinon 100) : consomme REGEN\_FOOD d'abord, puis s'assied et attend (plafond maxWaitSec, défaut REGEN\_MAX\_WAIT).

```
regenerate(90)
```

[Copier](#)

## Déplacement & voyage

**travelTo**(mapId) → bool

**travelTo**(x, y) → bool

**goToMapId**(mapId) → bool

Bloquant

Voyage multi-cartes vers une carte (id, coordonnées x, y ou chaîne "x,y") : BFS du world-graph + raccourci zaap / havre-sac si utile, sinon marche. Un id de carte périmé est résolu vers le nœud vivant à la même coordonnée. goToMapId est un alias.

```
travelTo(212861955)  -- Zaap Brakmar
travelTo(-15, 39)
```

Copier

**goToCellId**(cell) → bool

Bloquant

Déplace le personnage jusqu'à la cellule cell (0-559) de la carte courante (attend l'arrivée).

```
goToCellId(58)
```

Copier

**changeMap**(direction) → bool

Bloquant

Change de carte (3 tentatives) : direction "top", "bottom", "left", "right" (synonymes FR/EN acceptés), "top(123)" = sortie par cette cellule, "left|right" = une au hasard, ou téléport "zaap(id)", "zaapi(id)", "havenbag". Gère les sorties par bord comme par interactif/escalier.

```
changeMap("right")
changeMap("top(12)")  -- sortie par la cellule 12
changeMap("left|right")  -- une au hasard
```

Copier

## Interactifs & récolte

**gatherAll**() → bool

Bloquant

Perm.Gather

Récolte toutes les ressources récoltables de la carte (jusqu'à épuisement ou combat). Respecte ELEMENTS\_TO\_GATHER si défini (sinon toutes les vraies ressources).

```
gatherAll()
```

Copier

**gatherOne**() → bool

Bloquant

Perm.Gather

Récolte UNE ressource (la plus proche, filtrée par ELEMENTS\_TO\_GATHER) ; true si une a été tentée, false s'il n'y a plus rien. Pratique pour espacer les récoltes.

```
while gatherOne() do delay(1500) end
```

Copier

**goUseInteractive**(cell) → bool

Bloquant

Perm.Gather

Utilise l'interactif situé en cellule cell avec son skill par défaut (ressource, porte, escalier...) : se place à portée puis envoie l'action. Pour choisir le skill ou connaître l'issue exacte, voir map:useCell.

```
goUseInteractive(120)
```

Copier

## Combat (verbes globaux)

Démarrer un combat. Pour PILOTER le tour (positions, sorts, zones), voir onTurn (Cycle de vie & callbacks) et le module combat:.

**fight**() → bool

Bloquant

Perm.Fight

Attaque le groupe éligible le plus proche (filtres MIN/MAX\_MONSTERS, FORBIDDEN\_MONSTERS, FORCE\_MONSTERS ; régénère d'abord si MIN\_LIFE\_PERCENT) et attend la fin du combat, joué par le moteur ou par ton onTurn.

```
fight()
```

Copier

**fightCell**(cell) → bool

Bloquant

Perm.Fight

Comme fight() mais cible le groupe de monstres le plus proche de la cellule cell.

```
fightCell(305)
```

Copier

## Inventaire (verbes globaux)

Modèle uid→(gid, quantité) tenu par le moteur, seedé au login et maintenu en direct. Si l'inventaire n'a pas encore été reçu (aucun relog), `getInventoryItemCount` rend 0 et `use/drop` journalisent « inventaire pas encore reçu ». Voir aussi le module `inventory`: (équipement, niveau, poids...).

**`getInventoryItemCount(gid)`** → `number`

Quantité possédée de l'objet gid.

```
if getInventoryItemCount(16530) > 0 then print("J'ai un sac de Mandragore")
```

Copier

**`getItemName(gid)`** → `string`

Nom clair d'un gid (pour de jolis logs).

**`getKamas()`** → `number`

Kamas du personnage (-1 si inconnu).

**`useInventoryItem(gid [, n])`** → `bool`

**`useItem(gid [, n])`** → `bool`

Bloquant

Perm.Gather

Utilise / ouvre n exemplaires de l'objet gid (n omis = tous). Alias de `inventory:use`.

```
useInventoryItem(16533, 5)  -- ouvre 5 sacs de Millet
```

Copier

**`openBags()`** → `number`

Bloquant

Perm.Gather

Ouvre d'un coup tous les sacs de ressources du sac ; renvoie le nombre ouvert. Automatique avec `OPEN_BAGS`.

```
local n = openBags()
```

Copier

**deleteItem**(gid [, n]) → bool

**deleteInventoryItem**(gid [, n]) → bool

Perm.Gather

DÉTRUIT l'objet (≠ jeter au sol), sac uniquement ; n omis = tout le stack. Alias de inventory:delete ; automatique avec AUTO\_DELETE.

```
deleteItem(311)
```

Copier

**dropItem**(gid, count) → bool

**DropItem**(gid, count) → bool

Bloquant

Perm.Gather

Jette count de l'objet gid au sol (les deux casses existent).

```
dropItem(289, 100)
```

Copier

## Havre-sac

**enterHavenBag**() → bool

Bascule havre-sac : entre dans le havre-sac, ou en ressort si tu y es déjà (envoi de la demande ; true si envoyée). Le voyage l'utilise tout seul comme raccourci zaap. En trajet déclaratif : path = "havenbag" ou une étape map = "havenbag".

```
enterHavenBag()
```

Copier

## PNJ / dialogue

Dialoguer avec un PNJ et sélectionner ses réponses. Utile pour les quêtes, les hôteliers et les portes conditionnelles. Le module npc: (SnowBot) ajoute la réponse par identifiant, la banque par PNJ et la bascule vente / achat du HDV (même requête que goToHdv ; l'hôtelier s'ouvre par son interactif).

**talkNPC(npcId) → bool**

Bloquant

Ouvre le dialogue avec un PNJ de la carte et attend la QUESTION avant de rendre la main ( $\leq 4$  s, + 1,5 s pour la question). npcId = id d'acteur (négatif, map:getNpcs()[i].id) OU id de MODÈLE ( $> 0$ , comme dans les scripts SnowBot : map:getNpcs()[i].npcId), résolu sur la carte ; 0 = premier vrai PNJ de la carte (jamais un indice de chasse au trésor). false si le PNJ est introuvable (un modèle absent de la carte : tout de suite) ou ne répond pas.

```
local pnj = map:getNpcs()[1]
if pnj then talkNPC(pnj.id) end
```

Copier

**replyNPC(index) → bool**

Bloquant

Sélectionne la réponse visible n° index (1-based, plage 1..N) du dialogue ouvert, puis attend ( $\leq 2$  s) la question suivante ou la fermeture : un replyNPC enchaîné lit toujours les réponses de la BONNE question. true si la réponse est partie.  $\Delta$  C'est un INDEX, alors que npc:reply(n) de SnowBot prend un identifiant de réponse quand  $n > 0$ .

```
replyNPC(1)
```

Copier

**replyNPCAndChangeMap(index) → bool**

Bloquant

Sélectionne la réponse n° index (1-based) PUIS attend le changement de carte qu'elle déclenche (téléport/passage PNJ).

```
replyNPCAndChangeMap(1)
```

Copier

**getReplies() → number[]**

**global:getReplies() → number[]**

Identifiants des réponses de la question PNJ affichée ({} sans dialogue). Pour répondre par identifiant : npc:reply(id) ; par index : replyNPC(i).

```
for _, id in ipairs(getReplies()) do print("réponse " .. id) end
```

Copier

**leaveDialog()** → bool

Ferme le dialogue courant (PNJ, zaap, banque, HDV, coffre...) sans attendre.  
global:leaveDialog() / npc:leaveDialog() laissent en plus ~250 ms au serveur.

```
leaveDialog()
```

Copier

## Banque (coffre ouvert)

Dépôt / retrait sur un coffre DÉJÀ ouvert (banquier via npcBank ou bank:open(), coffre à code via map:lockedStorage). Voir aussi les modules bank: et exchange:.

**putAllItems()** → bool

Perm.Gather

Dépose TOUT le sac au coffre.

**putExistingItems()** → bool

Perm.Gather

Dépose uniquement les objets DÉJÀ présents au coffre (complète les piles existantes).

**getAllItems()** → bool

Perm.Gather

Retire TOUT le contenu du coffre vers le sac.

**getExistingItems()** → bool

Perm.Gather

Retire du coffre uniquement les objets déjà présents dans le sac.

**depositItem(gid [, n])** → bool

Perm.Gather

Dépose un objet précis par GUID (n ≤ 0 ou omis = tout le stack).

```
depositItem(289, 500)
```

Copier

## HDV — table sale:

API Hôtel de Vente en syntaxe colon sale:Verbe(...) (l'appel point sale.Verbe(...) marche aussi). Les actions (vente, reprice, retrait, achat, prix du marché) sont bloquantes et gâtées par la permission « Hôtel de Vente » ; les lectures de tes annonces (ItemsOnSale, GetItem\*, getGID, GetLots, AvailableSpace) sont instantanées et non contrôlées. Un HDV ouvert est fermé automatiquement avant tout déplacement.

### Prérequis : ouvrir le HDV dans le bon mode

goToHdv(mapId) voyage jusqu'à l'hôtelier et l'ouvre en mode VENTE (SellItem, EditPrice\*, RemoveItem, UpdateAllItems, repriceAll...). goToHdvBuy(mapId) l'ouvre en mode ACHAT (buyItem ; GetPricItem y renvoie le prix d'achat). Compat SnowBot : en Dofus 3, l'hôtelier s'ouvre par son INTERACTIF (goToHdv / goToHdvBuy font les deux : ouverture puis bascule) — ouvre-le (map:door, goUseInteractive ou map:useById ; il s'ouvre en mode achat), puis npc:npc(-1, 5) / npc:npc(-1, 6) envoie la MÊME requête que goToHdv pour le basculer en vente / achat (NpcGenericActionRequest, acteur -1 = l'hôtelier ; permission « Hôtel de Vente » ; le mode est reconnu tout seul). Les verbes échouent (log explicite) si le HDV n'est pas ouvert dans le mode attendu.

### Lot

Les arguments « lot » de GetPricItem / getAveragePricItem acceptent l'index SnowBot 1..4 (= 1, 10, 100, 1000) ou la taille 10/100/1000 ; 0 ou absent = 1. SellItem, EditPrice\*, RemoveItem et buyItem prennent une QUANTITÉ (taille du lot). Les accesseurs GetItem\* sont 1-based : for i = 1, sale:ItemsOnSale() (l'index 0 reste accepté comme alias du premier lot).

### Sentinelles (différences SnowBot)

Prix inconnus = -1 (SnowBot : 0) ; getGID d'un GUID qui n'est pas une de tes annonces = 0 (SnowBot : -1) ; buyItem avec prixMax = 0 = sans plafond (SnowBot n'achetait jamais). Teste > 0 plutôt que ~= 0.

```
goToHdv(mapId) → bool
```

Bloquant

Perm.Hdv

Global (pas dans sale:). Voyage jusqu'à la carte du HDV, ouvre l'hôtelier et bascule en mode VENTE. À appeler avant les verbes de vente.

```
goToHdv(HDV_MAP_ID)  -- carte de l'hôtelier cible
```

Copier

```
goToHdvBuy(mapId) → bool
```

Bloquant

Perm.Hdv

Global. Voyage jusqu'au HDV et ouvre l'hôtelier en mode ACHAT. À appeler avant sale:buyItem.

```
goToHdvBuy(HDV_MAP_ID)
```

Copier

```
sale:SellItem(GID, quantité, price) → bool
```

Bloquant

Perm.Hdv

Met en vente un lot de l'objet GID : quantité = taille du lot (1/10/100/1000, pas un index), au prix price. Résout le GID → objet du sac.

```
sale:SellItem(2303, 100, 5200)
```

Copier

```
sale:EditPriceByGID(GID, price, quantité) → bool
```

Bloquant

Perm.Hdv

Repositionne au prix price TOUTES tes annonces (GID, quantité). true aussi si tout était déjà au bon prix ; false si aucune annonce ne correspond ou si un reprice a échoué.

```
sale:EditPriceByGID(2303, 5193, 100)
```

Copier

```
sale:EditPrice(GUID, price, quantité) → bool
```

Bloquant

Perm.Hdv

Modifie le prix d'une annonce précise par son GUID (uid d'échange courant de l'annonce).

```
sale:EditPrice(sale:GetItemGUID(1), 5193, sale:GetItemQuantity(1))
```

Copier

`sale:RemoveItem(GUID, quantité) → bool`

Bloquant

Perm.Hdv

Retire de la vente l'annonce GUID (quantité du lot). Le lot repart dans ton sac.

```
sale:RemoveItem(sale:GetItemGUID(1), sale:GetItemQuantity(1))
```

Copier

`sale:getAveragePriceItem(GID [, lot]) → number`

Bloquant

Perm.Hdv

Interroge le HDV et renvoie le prix moyen UNITAIRE × taille du lot (lot = index 1..4 ou taille ; absent = 1). -1 si inconnu.

```
local moy100 = sale:getAveragePriceItem(2303, 3) -- lot de 100
```

Copier

`sale:GetPriceItem(GID [, lot]) → number`

Bloquant

Perm.Hdv

Prix d'un lot de GID : en mode ACHAT (goToHdvBuy), prix d'achat de l'offre ; sinon prix MIN des vendeurs. lot = index SnowBot 1..4 ou taille 10/100/1000. -1 si inconnu.

```
local minPrix = sale:GetPriceItem(2303, 100) -- ou sale:GetPriceItem(2303,
```

Copier

`sale:UpdateAllItems() → number`

Bloquant

Perm.Hdv

Sémantique SnowBot : rafraîchit le marché et repositionne TOUTES tes annonces au prix min - 1 (= repriceAll(1)). Ne remonte jamais un prix et ne se sous-cote jamais toi-même. Renvoie le nombre d'annonces repositionnées.

```
local n = sale:UpdateAllItems()
```

Copier